

南開大學
Nankai University

计算机学院

计算机组成原理 lab3 实验报告

寄存器堆实现

姓名：赵宇哲

学号：2411094

专业：计算机科学与技术

2026 年 4 月 30 日

目录

1 实验目的 **2**

2 实验内容说明 **2**

3 实验原理图 **2**

4 实验步骤 **4**

4.1 regfile.v 的修改 4

4.2 regfile_display.v 的修改 6

4.3 regfile.xdc 的修改 7

5 实验结果分析 **8**

5.1 上板验证 8

5.2 结果分析 11

6 总结感想 **11**

A 附录 **12**

A.1 regfile.v 12

A.2 regfile_display.v 13

A.3 regfile.xdc 19

1 实验目的

1. 熟悉并掌握 MIPS 计算机中寄存器堆的原理和设计方法。
2. 进一步理解寄存器堆异步读、同步写的实现方式。
3. 掌握利用 Verilog 对寄存器堆进行功能扩展的方法。
4. 在原实验基础上实现 2 位写使能控制，并完成高 16 位和低 16 位分段写入。
5. 增加专用寄存器，用于保存 1 号寄存器与 2 号寄存器数据之和。
6. 完成顶层显示模块与约束文件修改，并在实验箱上进行上板验证。

2 实验内容说明

本次实验在实验指导手册实验三“寄存器堆实现”的基础上，对寄存器堆进行了功能改进。主要内容如下：

1. 将原来的 1 位写使能信号 `wen` 扩展为 2 位，其中 `wen[1]` 控制高 16 位写入，`wen[0]` 控制低 16 位写入，从而实现对 32 位写数据的分段写入。
2. 在原有寄存器堆结构基础上，增加一个专用寄存器 `SUM12`，用于保存 1 号寄存器和 2 号寄存器当前数据之和。
3. 修改 `regfile.v` 模块，实现改进后的写控制逻辑、专用寄存器逻辑以及原有异步读同步写功能。
4. 修改顶层模块 `regfile_display.v`，使其能够完成输入选择、LCD 显示以及新增专用寄存器结果的展示。
5. 修改约束文件 `regfile.xdc`，重新分配写使能与输入选择相关的拨码开关和 LED 引脚。
6. 完成工程综合、实现与 bit 文件生成，并在实验箱上通过实际读写操作验证设计功能的正确性。

3 实验原理图

本次实验在原寄存器堆实验的基础上进行了功能扩展，改进后的寄存器堆仍采用异步读、同步写结构。寄存器堆内部包含 32 个 32 位寄存器，其中 0 号寄存器恒为 0。读端口通过读地址直接选择寄存器内容输出；写端口在时钟上升沿根据写地址和写使能信号完成数据写入。

与原实验相比，本实验主要进行了两点改进：第一，将原来的 1 位写使能信号 wen 扩展为 2 位，其中 wen[1] 控制写数据 wdata 的高 16 位写入，wen[0] 控制写数据 wdata 的低 16 位写入；第二，增加专用寄存器 SUM12，用于保存 1 号寄存器和 2 号寄存器当前数据之和。改进后的寄存器堆内部结构如图 3.1 所示。

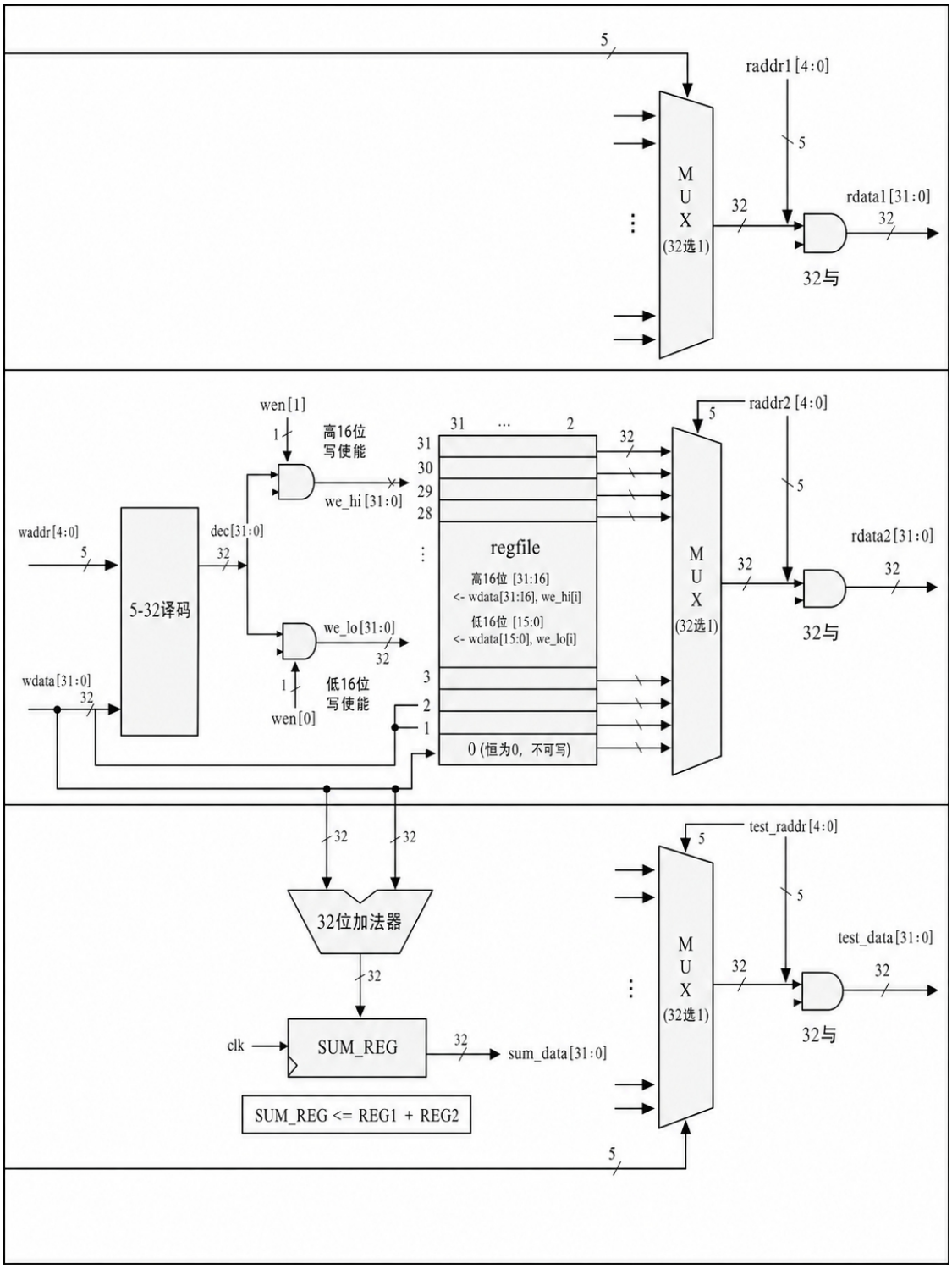


图 3.1: 改进后寄存器堆内部原理图

如图 3.1 所示，写地址 waddr 经 5-32 译码后选择目标寄存器。写数据 wdata 被分为高 16 位和低 16 位两部分，分别由 wen[1] 和 wen[0] 控制。当 wen[1] 有效时，将 wdata[31:16] 写入目标寄存器高 16 位；当 wen[0] 有效时，将 wdata[15:0] 写入目标寄存器低 16 位。若两位写使能同时有效，则完成完整 32 位写入。读端口 raddr1、raddr2 和调试读端口 test_addr 均采用异步读方式，可以直接根据地址读出对应寄存器的值。此外，SUM12 专用寄存器在时钟上升沿更新，用于保存 1 号寄存器与 2 号寄存器数据之和。

改进后的顶层结构如图 3.2 所示。顶层模块 regfile_display 调用底层寄存器堆模块 regfile，同时调用 LCD 触摸屏模块 lcd_module，用于完成数据输入和结果显示。

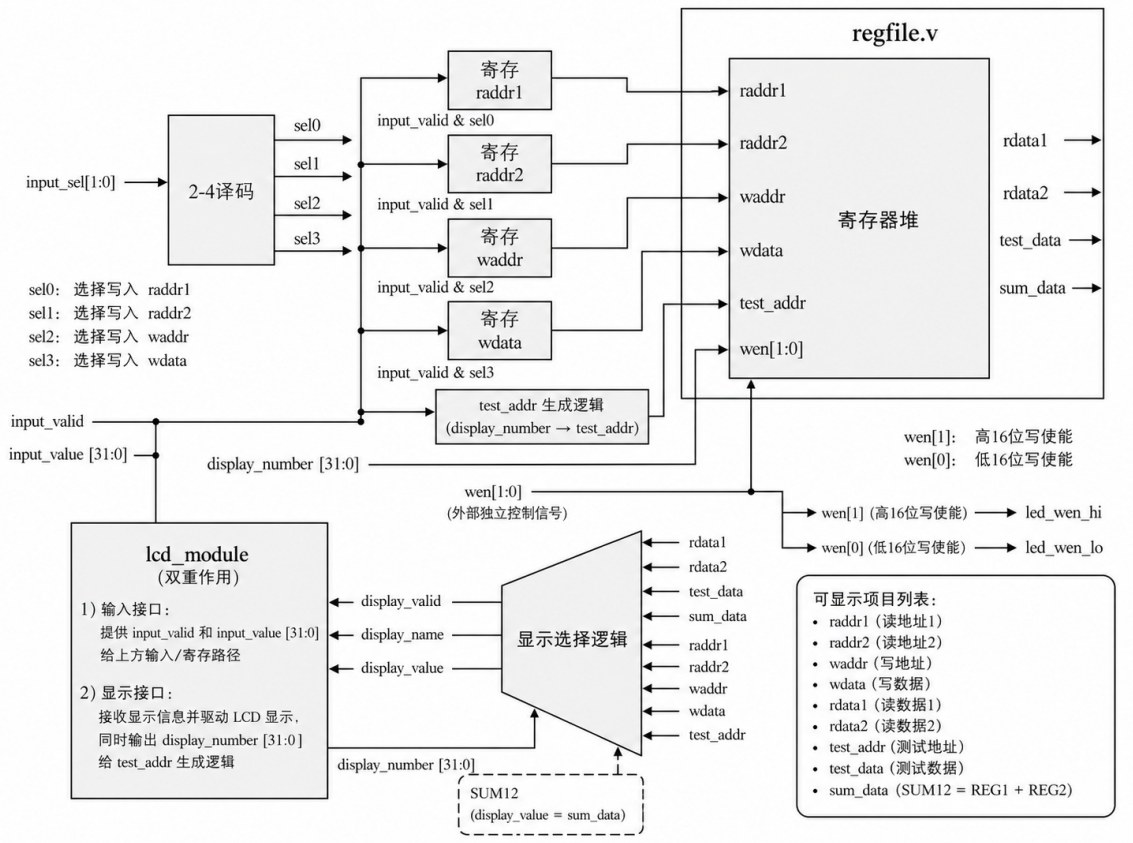


图 3.2: 改进后寄存器堆顶层原理图

如图 3.2 所示，拨码开关 wen[1:0] 用于控制高、低 16 位写使能，input_sel[1:0] 用于选择当前触摸屏输入的数据对象，包括读地址 1、读地址 2、写地址和写数据。LCD 触摸屏模块负责输入 32 位数据，并显示 RADD1、RDAT1、RADD2、RDAT2、WADDR、WDATA、32 个通用寄存器的值，以及新增的 SUM12 和 WEN___。LED 则用于指示当前写使能状态和输入选择状态，便于在实验箱上进行观察和调试。

4 实验步骤

本次实验主要完成了底层寄存器堆模块、顶层显示模块以及约束文件三部分的修改。通过对写使能信号、显示逻辑和引脚分配的调整，实现了寄存器堆分段写入以及专用寄存器显示等功能。

4.1 regfile.v 的修改

在 `regfile.v` 中，主要完成了两部分改动：一是将原来的 1 位写使能信号扩展为 2 位，分别控制高 16 位和低 16 位写入；二是增加专用寄存器 `sum12`，用于保存 1 号寄存器和 2 号寄存器中数据之和。

首先修改模块端口定义，将 `wen` 扩展为 2 位，并新增 `resetn` 和 `sum12` 接口：

```
1 module regfile(
2     input             clk,
```

```
    input              resetn,
    input      [1:0]   wen,
    input      [4:0]   raddr1,
    input      [4:0]   raddr2,
    input      [4:0]   waddr,
    input      [31:0]  wdata,
    output reg [31:0]  rdata1,
    output reg [31:0]  rdata2,
    input      [4:0]   test_addr,
    output reg [31:0]  test_data,
    output reg [31:0]  sum12
);
```

接着，在写入逻辑中根据 wen[1] 和 wen[0] 分别控制寄存器高 16 位和低 16 位写入：

```
always @(*) begin
    write_value = rf[waddr];

    if (wen[1])
        write_value[31:16] = wdata[31:16];

    if (wen[0])
        write_value[15:0]  = wdata[15:0];
end
```

然后在时钟上升沿完成同步写入，同时保证 0 号寄存器恒为 0：

```
always @(posedge clk or negedge resetn) begin
    if (!resetn) begin
        for (i = 0; i < 32; i = i + 1)
            rf[i] <= 32'd0;
        sum12 <= 32'd0;
    end
    else begin
        rf[0] <= 32'd0;
        if ((|wen) && (waddr != 5'd0))
            rf[waddr] <= write_value;
    end
end
```

为了实现新增的专用寄存器功能，在时钟上升沿对 1 号寄存器和 2 号寄存器的值求和，并保存到 sum12 中：

```
sum12 <= reg1_after_write + reg2_after_write;
```

读端口部分仍然保持原实验的异步读方式，例如：

```
always @(*) begin
    if (raddr1 == 5'd0)
        rdata1 = 32'd0;
```

```
    else
        rdata1 = rf[raddr1];
end
```

通过上述修改，寄存器堆在保留原有异步读、同步写结构的基础上，实现了分段写入和专用寄存器扩展功能。

4.2 regfile_display.v 的修改

在顶层模块 regfile_display.v 中，主要完成了三项修改：一是将写使能输入改为 2 位；二是增加对应的 LED 指示信号；三是在 LCD 上增加 SUM12 和 WEN__ 的显示。

首先修改顶层模块端口，将原来的单个 wen 扩展为 2 位，并增加两个 LED 输出：

```
input  [1:0] wen,
input  [1:0] input_sel,

output led_wen_h,
output led_wen_l,
output led_waddr,
output led_wdata,
output led_raddr1,
output led_raddr2,
```

然后修改 LED 指示逻辑，用于显示当前写使能状态以及输入对象选择状态：

```
assign led_wen_h  = wen[1];
assign led_wen_l  = wen[0];
assign led_raddr1 = (input_sel == 2'd0);
assign led_raddr2 = (input_sel == 2'd1);
assign led_waddr  = (input_sel == 2'd2);
assign led_wdata  = (input_sel == 2'd3);
```

接下来实例化改进后的寄存器堆模块，并将新增的 sum12 接口接入顶层：

```
wire [31:0] sum12;

regfile rf_module(
    .clk      (clk),
    .resetn   (resetn),
    .wen      (wen),
    .raddr1   (raddr1),
    .raddr2   (raddr2),
    .waddr    (waddr),
    .wdata    (wdata),
    .rdata1   (rdata1),
    .rdata2   (rdata2),
    .test_addr(test_addr),
    .test_data(test_data),
    .sum12    (sum12)
```

```
);
```

输入部分仍然保留原实验的四种输入模式，其中地址输入仍然只取触摸屏输入值的低 5 位：

```
always @(posedge clk) begin
    if (!resetn)
        raddr1 <= 5'd0;
    else if (input_valid && input_sel == 2'd0)
        raddr1 <= input_value[4:0];
end
```

最后，在 LCD 显示逻辑中增加 SUM12 和 WEN__ 的显示项：

```
6'd39 : begin
    display_valid <= 1'b1;
    display_name  <= "SUM12";
    display_value <= sum12;
end

6'd40 : begin
    display_valid <= 1'b1;
    display_name  <= "WEN__";
    display_value <= {30'd0, wen};
end
```

通过这些修改，顶层模块能够完成输入控制、结果显示以及新增专用寄存器功能的板上验证。

4.3 regfile.xdc 的修改

由于顶层模块端口发生了变化，因此约束文件 regfile.xdc 也需要同步修改。主要变化包括：将原来的单比特 wen 约束改为两位开关输入，并增加两个写使能 LED 的引脚分配。

写使能和输入选择开关的约束如下：

```
set_property PACKAGE_PIN AC21 [get_ports wen[1]]
set_property PACKAGE_PIN AD24 [get_ports wen[0]]
set_property PACKAGE_PIN AC22 [get_ports input_sel[1]]
set_property PACKAGE_PIN AC23 [get_ports input_sel[0]]
```

新增 LED 的约束如下：

```
set_property PACKAGE_PIN H7 [get_ports led_wen_h]
set_property PACKAGE_PIN D5 [get_ports led_wen_l]
set_property PACKAGE_PIN A3 [get_ports led_raddr1]
set_property PACKAGE_PIN A5 [get_ports led_raddr2]
set_property PACKAGE_PIN A4 [get_ports led_waddr]
set_property PACKAGE_PIN F7 [get_ports led_wdata]
```

5 实验结果分析

5.1 上板验证

在完成 `regfile.v`、`regfile_display.v` 和 `regfile.xdc` 的修改后，使用 Vivado 完成综合、实现与 bit 文件生成，并将生成的 bit 文件下载到 FPGA 实验箱中进行上板验证。

本次上板验证共进行了四组测试，分别对应整 32 位写入、低 16 位写入、高 16 位写入以及读端口和专用寄存器 `SUM12` 的联合验证。为方便板上调试，实验过程中首先确定了拨码开关与输入/写使能之间的对应关系，如图 5.3 所示。

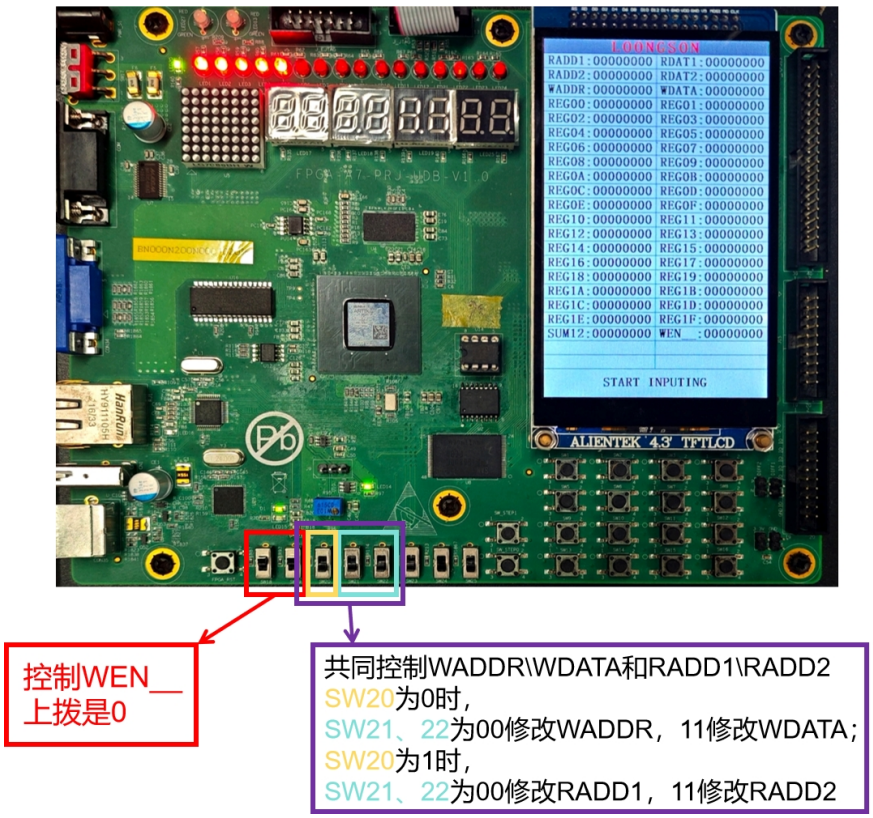


图 5.3: 实验箱拨码开关功能分配示意图

第一组实验验证整 32 位写入功能。设置写地址为 `0x00000001`，写数据为 `0x12345678`，并令 `WEN__ = 00000003`，此时 1 号寄存器被完整写入，实验结果如图 5.4 所示。

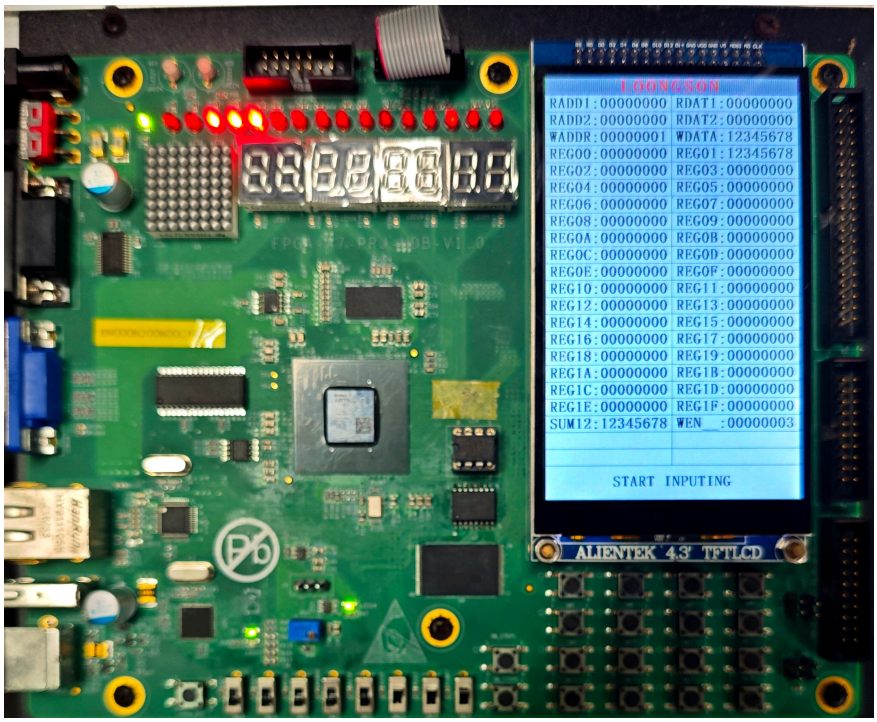


图 5.4: 整 32 位写入验证结果

第二组实验验证低 16 位写入功能。设置写地址为 0x00000002，写数据为 0x0000ABCD，并令 WEN__ = 00000001，此时仅对 2 号寄存器低 16 位进行写入，实验结果如图 5.5 所示。

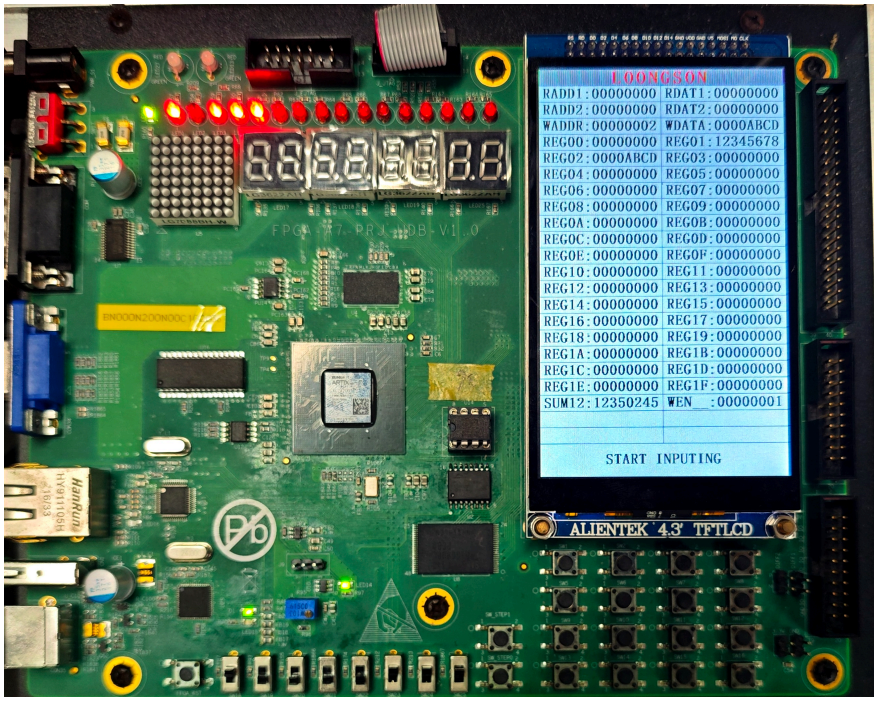


图 5.5: 低 16 位写入验证结果

第三组实验验证高 16 位写入功能。在第二组结果基础上，保持写地址为 0x00000002，将写数据修改为 0x56780000，并令 WEN__ = 00000002，此时仅对 2 号寄存器高 16 位进行写入，实验结果如图 5.6 所示。

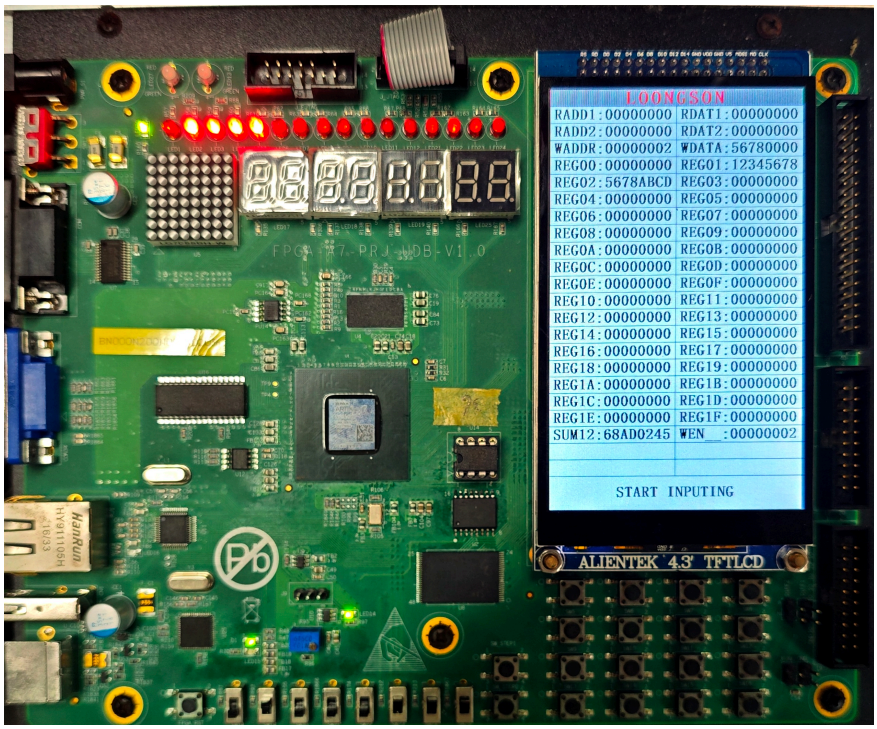


图 5.6: 高 16 位写入验证结果

第四组实验对读端口和新增专用寄存器 SUM12 进行联合验证。设置 RADD1 = 00000001、RADD2 = 00000002，并保持 WEN__ = 00000000，读取两个寄存器中的结果，同时观察 SUM12 的显示值，实验结果如图 5.7 所示。

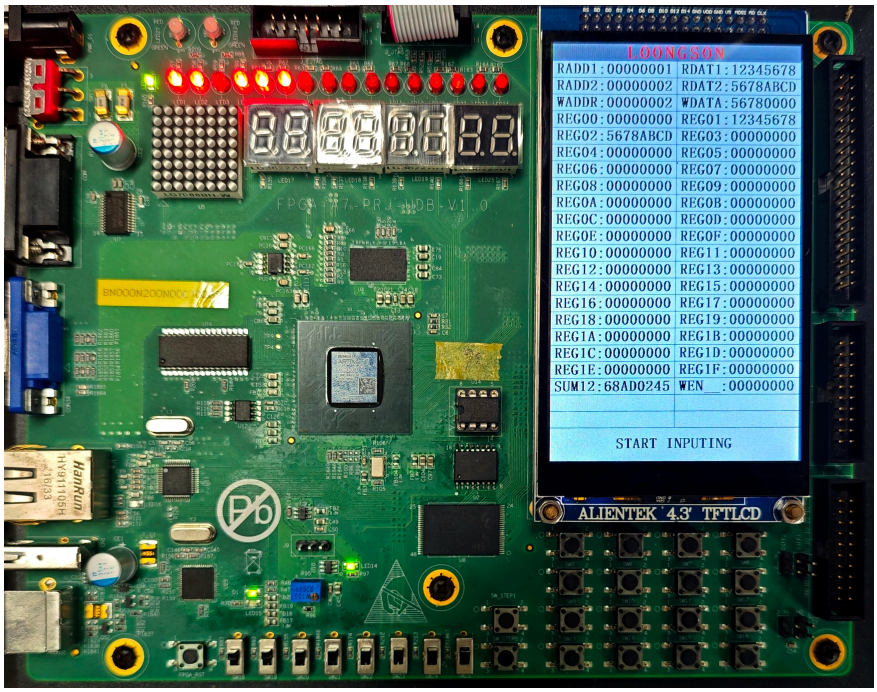


图 5.7: 读端口与专用寄存器 SUM12 验证结果

5.2 结果分析

对于第一组实验，由图 5.4 可见，此时屏幕上显示 WADDR = 00000001、WDATA = 12345678、WEN__ = 00000003，同时 REG01 = 12345678。这说明当两位写使能同时有效时，写数据的高 16 位和低 16 位都会被写入目标寄存器，因此 1 号寄存器被完整写成 0x12345678，整 32 位写入功能正确。

对于第二组实验，由图 5.5 可见，此时屏幕上显示 WADDR = 00000002、WDATA = 0000ABCD、WEN__ = 00000001，对应结果为 REG02 = 0000ABCD。这说明当仅低位写使能有效时，寄存器只写入低 16 位，而高 16 位保持不变，低 16 位分段写入功能正确。

对于第三组实验，由图 5.6 可见，此时屏幕上显示 WADDR = 00000002、WDATA = 56780000、WEN__ = 00000002，最终 REG02 = 5678ABCD。这说明在第二组已将低 16 位写为 ABCD 的基础上，本次仅将高 16 位写为 5678，从而得到最终结果 0x5678ABCD，说明高 16 位分段写入功能正确。

对于第四组实验，由图 5.7 可见，此时 RADD1 = 00000001，对应 RDAT1 = 12345678；RADD2 = 00000002，对应 RDAT2 = 5678ABCD；同时 SUM12 = 68AD0245。理论上，

$$0x12345678 + 0x5678ABCD = 0x68AD0245$$

与屏幕显示结果完全一致。这说明两个读端口均能够正确读出寄存器内容，且新增专用寄存器 SUM12 能够正确保存 1 号寄存器与 2 号寄存器数据之和。

综合以上四组实验结果可知，改进后的寄存器堆成功实现了以下功能：其一，原单比特写使能扩展为 2 位后，可以分别控制高 16 位和低 16 位写入；其二，整 32 位写入、低 16 位写入和高 16 位写入均能正确完成；其三，新增专用寄存器 SUM12 工作正常。说明本次实验的功能改进达到了预期要求。

6 总结感想

通过本次实验，我进一步理解了寄存器堆的基本工作原理，尤其是异步读、同步写的实现方式，以及读地址、写地址、写数据之间的配合关系。在原实验的基础上继续进行功能扩展，也让我对寄存器堆模块化设计的方法有了更直观的认识。

本次实验中，最主要的改进是将原来的单比特写使能扩展为 2 位写使能，从而分别控制高 16 位和低 16 位写入。通过实际修改代码和上板验证，我更加清楚地理解了分段写入的设计思路，也认识到在数字电路设计中，控制信号位宽变化往往会同时影响到底层功能模块、顶层显示模块以及约束文件配置，不能只修改其中某一部分。

此外，在本次实验中增加了专用寄存器 SUM12，用于保存 1 号寄存器和 2 号寄存器数据之和。通过这一功能的实现，我进一步体会到了在原有模块基础上增加新功能时，不仅要考虑功能本身的实现，还要考虑如何在顶层模块和实验箱显示界面中将结果正确展示出来。

在实验过程中，我也遇到了一些问题，例如拨码开关与输入对象之间的对应关系不够清晰、写使能状态切换时容易混淆、约束文件中端口名与顶层模块不一致等。这些问题在调试过程中花费了一定时间，但也正是在不断修改和验证的过程中，我对 Vivado 工程建立、代码综合、bit 文件生成以及实验箱上板调试流程有了更加完整的掌握。

总的来说，本次实验不仅完成了寄存器堆写控制与功能结构的改进，也让我进一步熟悉了从 Verilog 编码、顶层连接、约束配置到上板验证的完整实验流程，为后续更复杂的 CPU 相关实验打下了基础。

附录 A 附录

A.1 regfile.v

```
1  `timescale 1ns / 1ps
2
3  module regfile(
4      input              clk,
5      input              resetn,
6      input      [1:0]   wen,        // [1]: high16, [0]: low16
7      input      [4:0]   raddr1,
8      input      [4:0]   raddr2,
9      input      [4:0]   waddr,
10     input      [31:0]  wdata,
11     output reg [31:0]  rdata1,
12     output reg [31:0]  rdata2,
13     input      [4:0]   test_addr,
14     output reg [31:0]  test_data,
15     output reg [31:0]  sum12
16 );
17
18     reg [31:0] rf[31:0];
19     integer i;
20
21     reg [31:0] write_value;
22     reg [31:0] reg1_after_write;
23     reg [31:0] reg2_after_write;
24
25     always @(*) begin
26         write_value = rf[waddr];
27
28         if (wen[1])
29             write_value[31:16] = wdata[31:16];
30
31         if (wen[0])
32             write_value[15:0]  = wdata[15:0];
33
34         // 预测写完之后 reg1 / reg2 的值，便于同拍更新 sum12
35         reg1_after_write = rf[1];
36         if ((waddr == 5'd1) && (|wen) && (waddr != 5'd0))
37             reg1_after_write = write_value;
38
39         reg2_after_write = rf[2];
40         if ((waddr == 5'd2) && (|wen) && (waddr != 5'd0))
41             reg2_after_write = write_value;
42     end
43
44     // 同步写
45     always @(posedge clk or negedge resetn) begin
```

```
        if (!resetn) begin
            for (i = 0; i < 32; i = i + 1)
                rf[i] <= 32'd0;
            sum12 <= 32'd0;
        end
        else begin
            rf[0] <= 32'd0; // 0号寄存器恒为0

            if ((|wen) && (waddr != 5'd0))
                rf[waddr] <= write_value;

            // 专用寄存器: 保存1号和2号寄存器之和
            sum12 <= reg1_after_write + reg2_after_write;
        end
    end

    // 异步读端口1
    always @(*) begin
        if (raddr1 == 5'd0)
            rdata1 = 32'd0;
        else
            rdata1 = rf[raddr1];
    end

    // 异步读端口2
    always @(*) begin
        if (raddr2 == 5'd0)
            rdata2 = 32'd0;
        else
            rdata2 = rf[raddr2];
    end

    // 调试读端口
    always @(*) begin
        if (test_addr == 5'd0)
            test_data = 32'd0;
        else
            test_data = rf[test_addr];
    end

endmodule
```

A.2 regfile_display.v

```
`timescale 1ns / 1ps

//*************************************************************
// > 文件名: regfile_display.v
```

```
// > 描述   : 寄存器堆显示模块，调用 FPGA 板上的 IO 接口和触摸屏
// > 说明   : 在实验指导手册参考代码基础上完成如下修改：
// >          1）wen由1位改为2位，分别控制高16位/低16位写入
// >          2）LED增加两个写使能指示：led_wen_h / led_wen_l
// >          3）实例化新的regfile，增加sum12输出
// >          4）LCD新增显示SUM12和WEN__
//////////////////////////////////////////////////////////////////

module regfile_display(
    //时钟与复位信号
    input clk,
    input resetn, //后缀"n"代表低电平有效

    //=============================================================
    //【修改A】原代码：
    // input wen,
    // input [1:0] input_sel,
    // output led_wen,
    //
    // 现改为：
    // wen扩展为2位：
    //   wen[1] -> 高16位写使能
    //   wen[0] -> 低16位写使能
    //=============================================================
    input  [1:0] wen,
    input  [1:0] input_sel,

    //=============================================================
    //【修改A】原代码：
    // output led_wen,
    // output led_waddr,
    // output led_wdata,
    // output led_raddr1,
    // output led_raddr2
    //
    // 现改为两个写使能指示灯
    //=============================================================
    output led_wen_h,
    output led_wen_l,
    output led_waddr,  // 指示输入写地址
    output led_wdata,  // 指示输入写数据
    output led_raddr1, // 指示输入读地址1
    output led_raddr2, // 指示输入读地址2

    //触摸屏相关接口，不需要更改
    output lcd_rst,
    output lcd_cs,
    output lcd_rs,
    output lcd_wr,
```

```
    output lcd_rd,
    inout  [15:0] lcd_data_io,
    output lcd_bl_ctr,
    inout  ct_int,
    inout  ct_sda,
    output ct_scl,
    output ct_rstn
);

    //============================================================
    //【修改B】LED显示逻辑修改
    // 原代码:
    // assign led_wen = wen;
    // assign led_raddr1 = (input_sel==2'd0);
    // assign led_raddr2 = (input_sel==2'd1);
    // assign led_waddr  = (input_sel==2'd2);
    // assign led_wdata  = (input_sel==2'd3);
    //============================================================
    //-----{LED 显示}begin
    assign led_wen_h  = wen[1];
    assign led_wen_l  = wen[0];
    assign led_raddr1 = (input_sel == 2'd0);
    assign led_raddr2 = (input_sel == 2'd1);
    assign led_waddr  = (input_sel == 2'd2);
    assign led_wdata  = (input_sel == 2'd3);
    //-----{LED 显示}end

    //-----{调用寄存器堆模块}begin
    //寄存器堆增加一个调试读端口，用于在触摸屏上显示32个寄存器值
    wire [31:0] test_data;
    wire [4:0]  test_addr;

    reg  [4:0]  raddr1;
    reg  [4:0]  raddr2;
    reg  [4:0]  waddr;
    reg  [31:0] wdata;
    wire [31:0] rdata1;
    wire [31:0] rdata2;

    //============================================================
    //【修改C】实例化新的regfile，增加resetn、2位wen、sum12
    //============================================================
    wire [31:0] sum12;

    regfile rf_module(
        .clk      (clk),
        .resetn   (resetn),
        .wen      (wen),
        .raddr1   (raddr1),
```

```
        .raddr2   (raddr2),
        .waddr    (waddr),
        .wdata    (wdata),
        .rdata1   (rdata1),
        .rdata2   (rdata2),
        .test_addr(test_addr),
        .test_data(test_data),
        .sum12    (sum12)
    );
    //-----{调用寄存器堆模块}end

    //---------------------{调用触摸屏模块}begin--------------------//
    //-----{实例化触摸屏}begin
    //此小节不需要更改
    reg        display_valid;
    reg [39:0] display_name;
    reg [31:0] display_value;
    wire [5:0] display_number;
    wire       input_valid;
    wire [31:0] input_value;

    lcd_module lcd_module(
        .clk           (clk),    //10Mhz
        .resetn        (resetn),

        //调用触摸屏的接口
        .display_valid (display_valid),
        .display_name  (display_name),
        .display_value (display_value),
        .display_number(display_number),
        .input_valid   (input_valid),
        .input_value   (input_value),

        //lcd 触摸屏相关接口，不需要更改
        .lcd_rst       (lcd_rst),
        .lcd_cs        (lcd_cs),
        .lcd_rs        (lcd_rs),
        .lcd_wr        (lcd_wr),
        .lcd_rd        (lcd_rd),
        .lcd_data_io   (lcd_data_io),
        .lcd_bl_ctr    (lcd_bl_ctr),
        .ct_int        (ct_int),
        .ct_sda        (ct_sda),
        .ct_scl        (ct_scl),
        .ct_rstn       (ct_rstn)
    );
    //-----{实例化触摸屏}end

    //-----{从触摸屏获取输入}begin
```

```
152     //根据实际需要输入的数修改此小节
153     //建议对每一个数的输入，编写单独一个always块
154     //手册原版：32个寄存器显示在7~38号的显示块，故读地址为(display_number-7)
155     //这里保持不变
156     assign test_addr = display_number - 5'd7;
157 
158     // 当 input_sel 为 2'b00 时，表示输入数为读地址1，即 raddr1
159     always @(posedge clk) begin
160         if (!resetn) begin
161             raddr1 <= 5'd0;
162         end
163         else if (input_valid && input_sel == 2'd0) begin
164             raddr1 <= input_value[4:0];   // 保持手册原逻辑：地址取低5位
165         end
166     end
167 
168     // 当 input_sel 为 2'b01 时，表示输入数为读地址2，即 raddr2
169     always @(posedge clk) begin
170         if (!resetn) begin
171             raddr2 <= 5'd0;
172         end
173         else if (input_valid && input_sel == 2'd1) begin
174             raddr2 <= input_value[4:0];   // 保持手册原逻辑：地址取低5位
175         end
176     end
177 
178     // 当 input_sel 为 2'b10 时，表示输入数为写地址，即 waddr
179     always @(posedge clk) begin
180         if (!resetn) begin
181             waddr <= 5'd0;
182         end
183         else if (input_valid && input_sel == 2'd2) begin
184             waddr <= input_value[4:0];    // 保持手册原逻辑：地址取低5位
185         end
186     end
187 
188     // 当 input_sel 为 2'b11 时，表示输入数为写数据，即 wdata
189     always @(posedge clk) begin
190         if (!resetn) begin
191             wdata <= 32'd0;
192         end
193         else if (input_valid && input_sel == 2'd3) begin
194             wdata <= input_value;
195         end
196     end
197     //-----{从触摸屏获取输入}end
198 
199     //-----{输出到触摸屏显示}begin
200     //触摸屏上共有44块显示区域，可显示44组32位数据
```

```
201     //44块显示区域从1开始编号，编号为1~44
202     always @(posedge clk) begin
203         //============================================================
204         //【修改D-1】保留原手册逻辑：
205         // 7~38号显示块显示32个通用寄存器的值
206         //============================================================
207         if (display_number > 6'd6 && display_number < 6'd39) begin
208             display_valid <= 1'b1;
209             display_name[39:16] <= "REG";
210             display_name[15: 8] <= {4'b0011, 3'b000, test_addr[4]};
211             display_name[7 : 0] <= {4'b0011, test_addr[3:0]};
212             display_value <= test_data;
213         end
214         else begin
215             case (display_number)
216                 6'd1 : begin
217                     display_valid <= 1'b1;
218                     display_name  <= "RADD1";
219                     display_value <= raddr1;
220                 end
221 
222                 6'd2 : begin
223                     display_valid <= 1'b1;
224                     display_name  <= "RDAT1";
225                     display_value <= rdata1;
226                 end
227 
228                 6'd3 : begin
229                     display_valid <= 1'b1;
230                     display_name  <= "RADD2";
231                     display_value <= raddr2;
232                 end
233 
234                 6'd4 : begin
235                     display_valid <= 1'b1;
236                     display_name  <= "RDAT2";
237                     display_value <= rdata2;
238                 end
239 
240                 6'd5 : begin
241                     display_valid <= 1'b1;
242                     display_name  <= "WADDR";
243                     display_value <= waddr;
244                 end
245 
246                 6'd6 : begin
247                     display_valid <= 1'b1;
248                     display_name  <= "WDATA";
249                     display_value <= wdata;
```

```
                end

                //================================================
                //【修改D-2】新增显示块39：显示专用寄存器SUM12
                //================================================
                6'd39 : begin
                    display_valid <= 1'b1;
                    display_name  <= "SUM12";
                    display_value <= sum12;
                end

                //================================================
                //【修改D-3】新增显示块40：显示当前2位写使能
                //================================================
                6'd40 : begin
                    display_valid <= 1'b1;
                    display_name  <= "WEN__";
                    display_value <= {30'd0, wen};
                end

                default : begin
                    display_valid <= 1'b0;
                    display_name  <= 40'd0;
                    display_value <= 32'd0;
                end
            endcase
        end
    end
    //-----{输出到触摸屏显示}end
    //----------------------{调用触摸屏模块}end--------------------//

endmodule
```

A.3 regfile.xdc

```
# ============================================================
# regfile.xdc
# NOTE:
# [MODIFY] comments mark the places changed from the manual version
# To avoid encoding issues in Vivado, comments are kept in ASCII only
# ============================================================

# -------------------------
# clock
# -------------------------
set_property PACKAGE_PIN AC19 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports clk]

```

```
# -------------------------
# reset
# -------------------------
set_property PACKAGE_PIN Y3 [get_ports resetn]
set_property IOSTANDARD LVCMOS33 [get_ports resetn]

# ============================================================
# [MODIFY-1] switch mapping changed
# Manual version:
#   wen           -> 1 bit
#   input_sel[1]  -> 1 switch
#   input_sel[0]  -> 1 switch
#
# New version:
#   wen[1]        -> high 16-bit write enable
#   wen[0]        -> low 16-bit write enable
#   input_sel[1:0] unchanged in meaning
# ============================================================

# switches
set_property PACKAGE_PIN AC21 [get_ports wen[1]]
set_property IOSTANDARD LVCMOS33 [get_ports wen[1]]

set_property PACKAGE_PIN AD24 [get_ports wen[0]]
set_property IOSTANDARD LVCMOS33 [get_ports wen[0]]

set_property PACKAGE_PIN AC22 [get_ports input_sel[1]]
set_property IOSTANDARD LVCMOS33 [get_ports input_sel[1]]

set_property PACKAGE_PIN AC23 [get_ports input_sel[0]]
set_property IOSTANDARD LVCMOS33 [get_ports input_sel[0]]

# ============================================================
# [MODIFY-2] LED mapping changed
# Manual version:
#   led_wen
#   led_waddr
#   led_wdata
#   led_raddr1
#   led_raddr2
#
# New version:
#   led_wen_h     -> indicate wen[1]
#   led_wen_l     -> indicate wen[0]
#   led_raddr1
#   led_raddr2
#   led_waddr
#   led_wdata
# ============================================================
```

```
63
64  # leds
65  set_property PACKAGE_PIN H7 [get_ports led_wen_h]
66  set_property IOSTANDARD LVCMOS33 [get_ports led_wen_h]
67
68  set_property PACKAGE_PIN D5 [get_ports led_wen_l]
69  set_property IOSTANDARD LVCMOS33 [get_ports led_wen_l]
70
71  set_property PACKAGE_PIN A3 [get_ports led_raddr1]
72  set_property IOSTANDARD LVCMOS33 [get_ports led_raddr1]
73
74  set_property PACKAGE_PIN A5 [get_ports led_raddr2]
75  set_property IOSTANDARD LVCMOS33 [get_ports led_raddr2]
76
77  set_property PACKAGE_PIN A4 [get_ports led_waddr]
78  set_property IOSTANDARD LVCMOS33 [get_ports led_waddr]
79
80  set_property PACKAGE_PIN F7 [get_ports led_wdata]
81  set_property IOSTANDARD LVCMOS33 [get_ports led_wdata]
82
83  # ============================================================
84  # [MODIFY-3] LCD pins are NOT changed
85  # This block is kept the same as the manual reference
86  # ============================================================
87
88  # lcd
89  set_property PACKAGE_PIN J25 [get_ports lcd_rst]
90  set_property PACKAGE_PIN H18 [get_ports lcd_cs]
91  set_property PACKAGE_PIN K16 [get_ports lcd_rs]
92  set_property PACKAGE_PIN L8  [get_ports lcd_wr]
93  set_property PACKAGE_PIN K8  [get_ports lcd_rd]
94  set_property PACKAGE_PIN J15 [get_ports lcd_bl_ctr]
95  set_property PACKAGE_PIN H9  [get_ports {lcd_data_io[0]}]
96  set_property PACKAGE_PIN K17 [get_ports {lcd_data_io[1]}]
97  set_property PACKAGE_PIN J20 [get_ports {lcd_data_io[2]}]
98  set_property PACKAGE_PIN M17 [get_ports {lcd_data_io[3]}]
99  set_property PACKAGE_PIN L17 [get_ports {lcd_data_io[4]}]
100 set_property PACKAGE_PIN L18 [get_ports {lcd_data_io[5]}]
101 set_property PACKAGE_PIN L15 [get_ports {lcd_data_io[6]}]
102 set_property PACKAGE_PIN M15 [get_ports {lcd_data_io[7]}]
103 set_property PACKAGE_PIN M16 [get_ports {lcd_data_io[8]}]
104 set_property PACKAGE_PIN L14 [get_ports {lcd_data_io[9]}]
105 set_property PACKAGE_PIN M14 [get_ports {lcd_data_io[10]}]
106 set_property PACKAGE_PIN F22 [get_ports {lcd_data_io[11]}]
107 set_property PACKAGE_PIN G22 [get_ports {lcd_data_io[12]}]
108 set_property PACKAGE_PIN G21 [get_ports {lcd_data_io[13]}]
109 set_property PACKAGE_PIN H24 [get_ports {lcd_data_io[14]}]
110 set_property PACKAGE_PIN J16 [get_ports {lcd_data_io[15]}]
111 set_property PACKAGE_PIN L19 [get_ports ct_int]
```

```
112 set_property PACKAGE_PIN J24 [get_ports ct_sda]
113 set_property PACKAGE_PIN H21 [get_ports ct_scl]
114 set_property PACKAGE_PIN G24 [get_ports ct_rstn]
115
116 set_property IOSTANDARD LVCMOS33 [get_ports lcd_rst]
117 set_property IOSTANDARD LVCMOS33 [get_ports lcd_cs]
118 set_property IOSTANDARD LVCMOS33 [get_ports lcd_rs]
119 set_property IOSTANDARD LVCMOS33 [get_ports lcd_wr]
120 set_property IOSTANDARD LVCMOS33 [get_ports lcd_rd]
121 set_property IOSTANDARD LVCMOS33 [get_ports lcd_bl_ctr]
122 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[0]}]
123 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[1]}]
124 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[2]}]
125 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[3]}]
126 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[4]}]
127 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[5]}]
128 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[6]}]
129 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[7]}]
130 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[8]}]
131 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[9]}]
132 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[10]}]
133 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[11]}]
134 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[12]}]
135 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[13]}]
136 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[14]}]
137 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[15]}]
138 set_property IOSTANDARD LVCMOS33 [get_ports ct_int]
139 set_property IOSTANDARD LVCMOS33 [get_ports ct_sda]
140 set_property IOSTANDARD LVCMOS33 [get_ports ct_scl]
141 set_property IOSTANDARD LVCMOS33 [get_ports ct_rstn]
```